Week 01
Markov Decision Processes and Bandits

STUDY-RL — a self-paced Deep RL curriculum

October 20, 2026

Contents

Learning objectives 3

1 Motivation & intuition 4

2 Mathematical formulation 6
- 2.1 Multi-armed bandits . . . 6
- 2.2 Contextual bandits and LinUCB . . . 7
- 2.3 Markov decision processes . . . 8

3 Algorithms 9

4 Implementation walkthrough 11

5 Empirical results 12

6 Safety angle 14

7 Matching application 15

8 Exercises 16

9 Further reading 17

Learning objectives

Study companion. Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

Key Idea

By the end of this week you will be able to:

1. State the *Markov decision process* tuple $(\mathcal{S}, \mathcal{A}, \mathcal{T}, R, \gamma)$ precisely and explain when each component is observable.
2. Define the multi-armed bandit problem, derive the regret decomposition, and explain the logarithmic horizon dependence of the Lai–Robbins lower bound and the UCB1 upper bound.
3. Implement `epsilon-greedy`, UCB1, Thompson sampling, and Boltzmann/softmax exploration in NumPy and distinguish confidence-based optimism, posterior sampling, and action randomisation.
4. Generalise from MAB to the contextual bandit via **LinUCB** and relate it to off-policy evaluation in the synthetic insurance lead-broker matcher.
5. Distinguish *discounted* from *average-reward* objectives and compute the discounted return G_t for a finite trajectory by hand.
6. Articulate *why exploration is a safety problem* in any high-stakes deployment.

1 Motivation & intuition

Reinforcement learning is the study of *agents that learn by doing*. An agent receives observations from an environment, picks an action, gets a scalar reward, and the loop repeats. The agent's goal is to maximise the total reward it accumulates over time. Three ingredients distinguish RL from supervised learning:

The *environment* is the part of the world the agent cannot inspect, only probe.

1. *Feedback is evaluative, not instructive.* The environment tells the agent how good its choice was, not which choice was best.
2. *Decisions compound.* Today's action changes tomorrow's state, so the loss surface is non-stationary in a structured way.
3. *Behaviour shapes the data.* Data may be collected by the current policy (on-policy), another policy (off-policy), or a fixed historical logger (offline). The data distribution and the learning target must therefore be distinguished.

Intuition. If supervised learning is "here is the right answer, copy it", RL is "commit to an answer, find out later, and don't poison the next ten questions while you are figuring it out".

From an ML/DL background. If you have trained a neural net on ImageNet, you have already seen the easy half of RL: function approximation. The hard half is the *data-generation loop*. In RL you choose the dataset as you go — and choosing it badly is what causes most real-world failures, not bad gradients.

Bandits first. Before we bring in transitions, we strip the problem of state and look at the *multi-armed bandit (MAB)*: K slot machines, each with an unknown reward distribution; pick one per round; how fast can you discover the best? Bandits provide a useful starting model for matching decisions. "Which broker should I show this lead to?" can be modelled as a contextual bandit; choosing an ordered shortlist can be modelled as a slate bandit. Whether a real matcher is well modelled as a bandit depends on delayed outcomes, capacity constraints, interference, and whether current decisions change future opportunities.

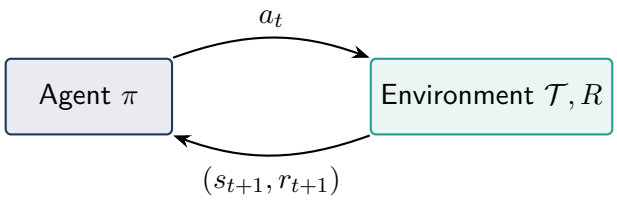


Figure 1.1: The agent–environment loop. Bandits are the special case where the state never changes, so the only choice is the action.

Common pitfall

Reward is not a loss. A negative reward is a *penalty*, not a signal that you should "learn less". Beginners flip the sign of the reward when porting supervised code, train a policy that minimises return, and conclude that "RL doesn't work". Always check the sign convention

before debugging anything else.

2 Mathematical formulation

2.1 Multi-armed bandits

Definition 2.1.1 (Stochastic multi-armed bandit)**.** A *stochastic K-armed bandit* is a tuple $(\mathcal{A}, \{\nu_a\}_{a\in\mathcal{A}})$ where $\mathcal{A} = \{1, \dots, K\}$ is the set of arms and each ν_a is a probability distribution over $\mathbb{R}$ with mean $\mu_a = \mathbb{E}_{R\sim\nu_a}[R]$. At each round $t = 1, 2, \dots$ the learner selects $A_t \in \mathcal{A}$ and observes a fresh reward from ν_{A_t}, conditionally independent of the past given the selected arm.

Let $\mu^\star = \max_a \mu_a$ and $\Delta_a = \mu^\star - \mu_a \geq 0$. The *cumulative regret* after T rounds is

$$R_T \triangleq T\mu^\star - \mathbb{E}\left[\sum_{t=1}^{T} R_t\right] = \sum_{a=1}^{K} \Delta_a \mathbb{E}[N_a(T)], \tag{2.1}$$

where $N_a(T) = \sum_{t=1}^{T} \mathbf{1}\{A_t = a\}$. The decomposition on the right is the foundation of every regret proof in this lecture: *regret is the total time spent on sub-optimal arms, weighted by how sub-optimal each is.*

Theorem 2.1.2 (Lai–Robbins lower bound, 1985)**.** *For a uniformly efficient algorithm (expected regret $o(T^c)$ for every $c > 0$ on every Bernoulli instance), with means in $(0, 1)$ and any suboptimal arm a,*

$$\liminf_{T\to\infty} \frac{\mathbb{E}[N_a(T)]}{\log T} \geq \frac{1}{\mathrm{KL}(\nu_a, \nu^\star)},$$

hence $R_T \geq (1 + o(1)) \sum_{a:\Delta_a>0} \frac{\Delta_a}{\mathrm{KL}(\nu_a,\nu^\star)} \log T$.

Sketch. Change-of-measure argument. If the algorithm rarely pulled a under the true problem, it could not distinguish it from a perturbed problem where a has mean $\mu^\star + \epsilon$, with perturbed reward law ν'_a. The KL between the two interaction-history laws is $\mathbb{E}[N_a(T)]\mathrm{KL}(\nu_a, \nu'_a)$ because only this arm's reward law changes. Uniform efficiency forces this information to grow at least as $(1 - o(1)) \log T$. Taking ϵ down to zero yields the Bernoulli denominator in the theorem. □

Theorem 2.1.3 (UCB1, Auer–Cesa-Bianchi–Fischer 2002)**.** *Suppose stationary independent arm rewards lie in $[0, 1]$ almost surely, and pull every arm once initially. The UCB1 policy*

$$A_{t+1} = \arg\max_a \ \hat{\mu}_a(t) + \sqrt{\frac{2\log t}{N_a(t)}}$$

satisfies

$$R_T \ \leq \ \sum_{a:\Delta_a>0} \frac{8\log T}{\Delta_a} \ + \ \left(1 + \tfrac{\pi^2}{3}\right) \sum_{a:\Delta_a>0} \Delta_a. \tag{2.2}$$

Sketch with explicit count union. Fix a suboptimal arm a and horizon $T \geq K$. Let $\ell = \lceil 8\log T/\Delta_a^2 \rceil$ and $c_{t,n} = \sqrt{2\log t/n}$. After t total pulls, the two sample counts are adaptive. For fixed counts $n_*, n_a \leq t$, Hoeffding bounds each of the one-sided bad events

$$\hat{\mu}_{*,n_*} \leq \mu^* - c_{t,n_*}, \qquad \hat{\mu}_{a,n_a} \geq \mu_a + c_{t,n_a}$$

by t^{-4}. If neither occurs and $n_a \geq \ell$, then the optimal arm's index is strictly above μ^*, while arm a's index is strictly below $\mu_a + 2c_{t,n_a} \leq \mu^*$, so arm a cannot be selected. Union-bound over both possible counts, not just the suboptimal arm's count:

$$\mathbb{P}\{A_{t+1} = a, N_a(t) \geq \ell\} \leq \sum_{n_*=1}^{t} \sum_{n_a=\ell}^{t} 2t^{-4} \leq 2t^{-2}.$$

There are at most ℓ pulls before the threshold. Hence $\mathbb{E}[N_a(T)] \leq 8 \log T/\Delta_a^2 + 1 + \pi^2/3$. Multiply by each gap and sum to obtain (2.2). □

Remark. Compare like quantities: the regret coefficients are $8/\Delta_a$ and $\Delta_a/\mathrm{KL}(\nu_a, \nu^*)$, whereas the pull-count coefficients are $8/\Delta_a^2$ and $1/\mathrm{KL}(\nu_a, \nu^*)$. For fixed interior Bernoulli mean $q = \mu^* \in (0, 1)$ and small gap, $\mathrm{KL}(\mathrm{Bern}(q - \Delta), \mathrm{Bern}(q)) = \Delta^2/[2q(1-q)] + O(\Delta^3)$. Thus both regret coefficients scale as $1/\Delta$ in this regime, with different constants. This is not a uniform constant-factor statement as means approach the boundaries. KL-UCB can attain the Bernoulli asymptotic lower-bound coefficient under its stated assumptions.

Theorem 2.1.4 (Thompson sampling: scope of the original bound)**.** *With independent stationary Bernoulli arms, interior means, a unique best arm and independent Beta*$(1, 1)$ *priors, Agrawal–Goyal (COLT 2012) prove logarithmic expected regret. For two arms their bound is* $O(\log T/\Delta + 1/\Delta^3)$*; their general-K result has a looser gap dependence. Later analyses improve the gap dependence. Do not attribute the later* $\sum_a \log T/\Delta_a$ *bound to the original general-K theorem.*

Proof roadmap. Control both suboptimal posterior samples and waiting for an optimistic sample from the optimal arm. Concentration of only the suboptimal posterior is insufficient. Bayesian information-ratio bounds average over a prior and are a different guarantee from fixed-instance frequentist regret. See https://proceedings.mlr.press/v23/agrawal12.html.

Boltzmann / softmax exploration. $\mathbb{P}(A_t = a) = \exp(\hat{\mu}_a/\tau)/\sum_b \exp(\hat{\mu}_b/\tau)$. Temperature $\tau \to 0$ recovers the greedy policy; $\tau \to \infty$ uniformises. Boltzmann exploration has no $\log T$ regret guarantee in general because it can stop exploring sub-optimal arms whose estimates are far below the optimum, even when the estimates are very uncertain. It is, however, ubiquitous in deep RL (SOFT ACTOR-CRITIC uses an entropy-regularised analogue).

2.2 Contextual bandits and LinUCB

Definition 2.2.1 (Stochastic contextual bandit)**.** At round t the learner observes a context $X_t \in \mathbb{R}^d$ drawn i.i.d. from an unknown distribution, selects $A_t \in \mathcal{A}$, and observes $R_t = \langle X_t, \theta^\star_{A_t} \rangle + \eta_t$ where η_t is σ-sub-Gaussian conditional on the past, and the $\theta^\star_a \in \mathbb{R}^d$ are unknown. *Linear* bandits have a single shared $\theta^\star$; the disjoint-arm variant has one per arm.

LinUCB maintains, per arm,

$$A_a^{(t)} = \lambda I + \sum_{s \leq t : A_s = a} X_s X_s^\top, \quad b_a^{(t)} = \sum_{s \leq t : A_s = a} R_s X_s, \quad \hat{\theta}_a^{(t)} = (A_a^{(t)})^{-1} b_a^{(t)},$$

and acts by

$$A_{t+1} = \arg\max_a \langle X_{t+1}, \hat{\theta}_a^{(t)} \rangle + \alpha \sqrt{X_{t+1}^\top (A_a^{(t)})^{-1} X_{t+1}}. \tag{2.3}$$

The bonus is the radius of the elliptical confidence region for $\theta^\star_a$, projected onto the current context. Abbasi-Yadkori, Pál & Szepesvári (2011) prove a problem-independent $\tilde{O}(d\sqrt{T})$ regret bound for the single-$\theta^\star$ case under self-normalised concentration.

2.3 Markov decision processes

Definition 2.3.1 (Finite MDP)**.** A finite Markov decision process is a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{T}, R, \gamma)$:

- $\mathcal{S}$ a finite state space; $\mathcal{A}$ a finite action space.
- $\mathcal{T} : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ the transition kernel, $\mathcal{T}(s' \mid s, a)$.
- $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ the conditional mean reward function. A joint kernel $p(s', r \mid s, a)$ describes stochastic rewards; assume sampled rewards satisfy $|r| \le R_{\max}$.
- $\gamma \in [0, 1)$ the discount factor (for episodic MDPs we allow $\gamma = 1$ when the return is integrable, for example with bounded rewards and finite expected episode length under the policies considered).

The *Markov property* is the statement that, given S_t and A_t, the random variables (S_{t+1}, R_{t+1}) are independent of all earlier history.

Definition 2.3.2 (Return)**.** For a trajectory $(S_0, A_0, R_1, S_1, A_1, R_2, \dots)$, the *discounted return* is $G_t \triangleq \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$.

Discounted vs. average reward. Two scalar criteria coexist in the literature:

- *Discounted*: $J_\gamma(\pi) = \mathbb{E}_\pi[\sum_{t \ge 0} \gamma^t R_{t+1}]$. Algorithms enjoy contraction at rate γ (Week 2) but the answer depends on the somewhat arbitrary choice of γ.
- *Average reward*: $J_{\text{avg}}(\pi) = \lim_{T \to \infty} \frac{1}{T} \mathbb{E}_\pi[\sum_{t=1}^{T} R_t]$. Independent of an arbitrary discount and is the right criterion for long-running matching marketplaces, but the Bellman equation involves a *differential* value $h(s)$ and the bias-optimality theory is more delicate (Puterman 1994, Ch. 8).

Discounts in a synthetic experiment are design choices. State the time step and outcome horizon, then compare policies under several discounts; no fixed number is justified merely by naming an application.

Common pitfall

Don't confuse γ with a regulariser. Lowering γ does not make learning "easier" — it changes the objective. A myopic agent ($\gamma \to 0$) is solving a different problem from a far-sighted agent and will pick different actions in non-trivial MDPs.

3 Algorithms

Algorithm 1: ϵ-greedy bandit.

Input: horizon T, exploration parameter ϵ.

Initialise $\hat{\mu}_a \leftarrow 0$, $N_a \leftarrow 0$ for all a.;

for $t = 1, \dots, T$ **do**

  With probability ϵ, pick A_t uniformly at random;

  Otherwise $A_t \leftarrow \arg\max_a \hat{\mu}_a$;

  Observe $R_t \sim \nu_{A_t}$; update $N_{A_t} \mathrel{+}= 1$, $\hat{\mu}_{A_t} \mathrel{+}= \frac{1}{N_{A_t}}(R_t - \hat{\mu}_{A_t})$;

end

Algorithm 2: UCB1.

Input: horizon T, exploration constant $c = 2$.

Pull each arm once to initialise; then for $t = K+1, \dots, T$:;

$A_t \leftarrow \arg\max_a \hat{\mu}_a + \sqrt{c \log t / N_a}$;

Observe R_t; update $\hat{\mu}_a$, N_a incrementally as above.;

Algorithm 3: Beta-Bernoulli Thompson sampling.

Input: Beta$(1,1)$ priors $\{\alpha_a, \beta_a\}$.

for $t = 1, \dots, T$ **do**

  Sample $\tilde{\mu}_a \sim \text{Beta}(\alpha_a, \beta_a)$ for each a;

  $A_t \leftarrow \arg\max_a \tilde{\mu}_a$;

  Observe $R_t \in \{0, 1\}$; $\alpha_{A_t} \mathrel{+}= R_t$, $\beta_{A_t} \mathrel{+}= 1 - R_t$;

end

Algorithm 4: LinUCB (disjoint arms).

Input: ridge $\lambda > 0$, bonus $\alpha > 0$.

For each arm a: $A_a \leftarrow \lambda I_d$, $b_a \leftarrow 0$.;

for $t = 1, \dots, T$ **do**

  Observe context X_t;

  For each a: $\hat{\theta}_a \leftarrow A_a^{-1} b_a$; $u_a \leftarrow X_t^\top \hat{\theta}_a + \alpha \sqrt{X_t^\top A_a^{-1} X_t}$;

  $A_t \leftarrow \arg\max_a u_a$; observe R_t;

  $A_{A_t} \mathrel{+}= X_t X_t^\top$; $\quad b_{A_t} \mathrel{+}= R_t X_t$;

end

Example 3.0.1 (Worked update)**.** Suppose two Bernoulli arms with current Beta posteriors $(\alpha_1, \beta_1) = (3, 2)$ and $(\alpha_2, \beta_2) = (1, 4)$. A single Thompson sample might return $\tilde{\mu}_1 = 0.62$ and $\tilde{\mu}_2 = 0.15$, so the agent pulls arm 1. If the reward is $R_t = 0$, the posterior becomes $(\alpha_1, \beta_1) = (3, 3)$. Its mean falls from 0.6 to 0.5, while its variance falls from 0.04 to $1/28 \approx 0.0357$. A disappointing observation need not increase posterior uncertainty.

Complexity. Per-round cost is $O(K)$ for ϵ-greedy/UCB1/TS; $O(Kd^2)$ for LinUCB because of the A_a^{-1} multiplication (Sherman–Morrison brings the matrix update to $O(d^2)$).

Common pitfall

Logarithm-of-zero. UCB1 must be initialised by pulling each arm at least once, otherwise the $\sqrt{2 \log t / N_a}$ bonus is infinite or undefined and the first K rounds are determined by floating-point quirks.

4 Implementation walkthrough

The mini-project lives in `project/`. The agents are deliberately written as small dataclasses so you can inspect their internal state in the notebook.

```python
# project/agent.py (excerpt)
@dataclass
class UCB1Agent:
    K: int; c: float = 2.0; seed: int = 0
    def __post_init__(self):
        self.rng = np.random.default_rng(self.seed)
        self.counts = np.zeros(self.K, dtype=int)
        self.means  = np.zeros(self.K, dtype=float)
    def select(self, t):
        unplayed = np.flatnonzero(self.counts == 0)
        if unplayed.size:
            return int(unplayed[0])
        bonuses = np.sqrt(self.c * np.log(max(t, 2)) / self.counts)
        ucb = self.means + bonuses
        return int(self.rng.choice(np.flatnonzero(ucb == ucb.max())))
    def update(self, a, r):
        self.counts[a] += 1
        self.means[a]  += (r - self.means[a]) / self.counts[a]
```

The synthetic insurance-style contextual environment (`project/env.py`) generates a non-intercept feature vector with $X_{t,j} \sim \mathcal{N}(0, 0.7^2)$ (with $X_{t,1} \equiv 1$ as an intercept slot) and computes the binding probability via a sigmoid of $\langle X_t, \theta_a^\star \rangle + b_a$. The latent $\theta_a^\star$ matrix is drawn once per environment construction, using its seed. It is fixed within that environment; changing the seed changes both the latent instance and the generated observations. The sigmoid reward means are not exactly linear, so LinUCB here is an approximation rather than a direct application of its linear-model regret guarantee.

Reproducibility. Every agent receives a deterministic seed; the environment uses an independent generator so reward noise does not co-vary with agent randomness. This isolates the variance you see in figures to environment noise plus exploration noise.

5 Empirical results

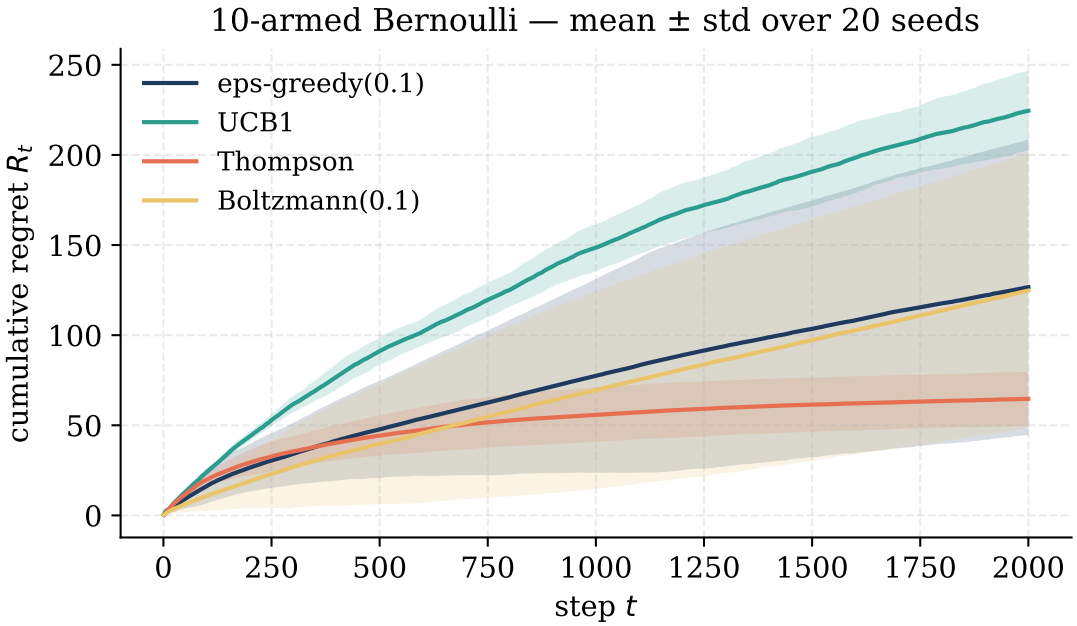


Figure 5.1: 10-armed Bernoulli testbed. Means $\in [0.1, 0.7]$ in steps of 0.05, $T = 2000$, 20 seeds. Thompson sampling clearly dominates; UCB1 ($c = 2$) is over-exploratory in the first ~ 500 rounds but its slope flattens sub-linearly afterwards. Boltzmann ($\tau = 0.1$) is competitive but hyper-parameter sensitive.

Intuition. The slope of regret *after* the burn-in phase is the diagnostic you care about. A bandit algorithm with steep early regret but a clean log-linear tail is much safer than one with a flat early curve that turns linear in the long run — the latter is silently mis-calibrating its uncertainty.

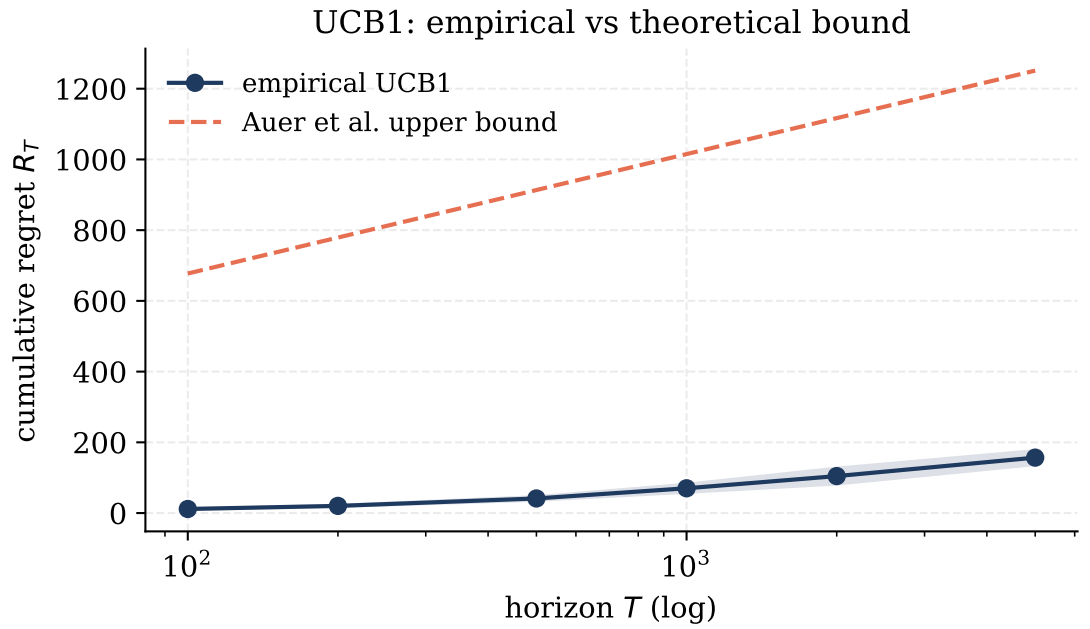


Figure 5.2: Empirical UCB1 regret vs the Auer et al. theoretical upper bound (2.2) on a 4-arm Bernoulli with gaps $\Delta \in \{0.1, 0.2, 0.3\}$. The empirical curve sits well below the bound, which is loose by a constant but matches the slope.

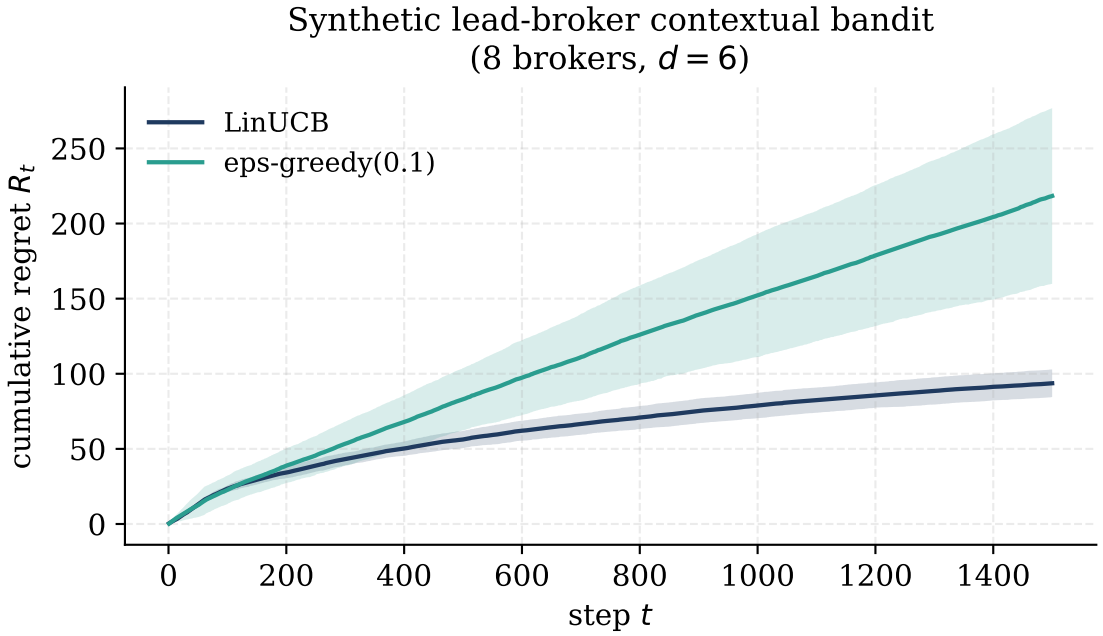


Figure 5.3: synthetic insurance lead-broker contextual bandit (8 brokers, $d = 6$). LinUCB exploits the context structure and achieves sub-linear regret; ϵ-greedy ignores the context and grows linearly.

6 Safety angle

Safety angle

Exploration in a high-stakes domain. Pure ϵ-greedy is the most popular reinforcement-learning idea and the most dangerous one to deploy in matching. Three concrete failure modes:

1. *Uniform exploration on a sensitive feature.* If your context includes a protected attribute (e.g. postcode in the UK, which proxies for ethnicity), an ϵ-greedy router sprays leads uniformly over brokers, including some who are systematically worse on certain cohorts. Over a few weeks of logs this looks like demographic-parity violation, and you cannot disentangle exploration noise from broker bias.
2. *Unbounded posterior pulls.* Thompson sampling will, with positive probability, sample from the tail of a flat posterior and pick a broker with no track record. Without a *floor* on the number of observations per arm, that one tail sample can route a high-value commercial lead to a broker who has never handled commercial. This is not a bug in TS — it is the cost of exploration — and the right response is a domain-specific safety filter, not abandoning TS.
3. *Mis-specified rewards.* If the operator logs binding as the reward but the platform is paid on revenue, you maximise binds and leave revenue on the table; worse, you train brokers to bind cheap and cancel later, gaming the optimised metric. Reward design is a safety problem.

Mitigations. **(i)** Cap ϵ-greedy with a *minimum-traffic gate*: a broker must have $N_a \geq n_{\min}$ before being eligible for high-value leads, and the first $n_{\min}$ pulls come only from low-value leads with bounded downside. **(ii)** Use *shielded exploration*: a hard rules layer rejects any action a regulator would flag, applied *after* the bandit chooses (see Alshiekh et al., AAAI 2018; Achiam et al., CPO, ICML 2017). **(iii)** Log the probability of the final executed action, including any filtering and renormalisation, not just the proposed action. Missing propensities prevent straightforward IPS; model-based evaluation then needs separate assumptions.

Citations. Alshiekh, Bloem, Ehlers, Könighofer, Niekum, Topcu — Safe RL via Shielding, AAAI 2018 (arXiv:1708.08611); Joachims, Swaminathan, de Rijke — Deep Learning with Logged Bandit Feedback, ICLR 2018.

7 Matching application

Matching application

synthetic insurance lead-broker matcher as a contextual bandit. A lead $x \in \mathbb{R}^d$ encodes postcode hash, vehicle / property descriptors, prior claim summary, derived text-embedding of the free-text question, and a constant intercept. Action $a \in \{1, \ldots, K\}$ is the broker; reward $r \in \{0, 1\}$ is the binding event observed days later.

When is a bandit sufficient? The one-step model assumes that routing choices do not materially change future opportunities. Adding the current queue to the context helps predict immediate outcomes, but does not remove the effect of today's action on tomorrow's queue. If those effects matter to the objective, include them in a sequential model. Delayed feedback is a separate issue (Chapelle 2014; see Week 11).

Mapping the math

X_t	lead feature vector at time t
A_t	broker chosen by the policy
$\theta_a^\star$	latent "binding affinity" of broker a across feature dimensions
R_t	1 if a policy is bound within the SLA window, 0 otherwise
π_b	current production routing policy (e.g. rule-based)
π	candidate new policy (LinTS / LinUCB)

Experiment to run. Compare LinUCB and ϵ-greedy on the same synthetic instances, reporting cumulative regret across seeds (Figure 5.3). A synthetic improvement does not establish an effect size on real routing data, where true regret is generally unobserved.
The ATS equivalent (Week 1 is also relevant to candidate ranking) is the same bandit with X_t = candidate embedding, A_t = job requisition, R_t = interview-yes. Position-bias correction (Joachims–Swaminathan–Schnabel 2017) is one ingredient of the CLTR pipeline studied in Week 12.

8 Exercises

Full versions with hints and graded tests are in `workbook.ipynb`. Reference solutions are in `project/solutions.py`.

Ex. 1 ($\star$). Implement the incremental sample-average update $Q_n = Q_{n-1} + \frac{1}{n}(R_n - Q_{n-1})$ and verify it equals $\bar{R}_n$ to floating-point precision.

Ex. 2 ($\star$). Run ϵ-greedy on a 3-arm Bernoulli with gaps $(0.6, 0.3, 0.0)$ for $T = 200$. Plot cumulative regret. Why is it non-decreasing?

Ex. 3 ($\star\star$). Implement the UCB1 bonus $\sqrt{2 \log t / N_a}$. Verify that the bonus shrinks as $N_a \to \infty$ and is infinite for unpulled arms.

Ex. 4 ($\star\star$). Show that after observing s successes and $n - s$ failures on a Beta(1,1) prior, the posterior is Beta$(1 + s, 1 + n - s)$. Implement the update.

Ex. 5 ($\star\star$). Bake off ϵ-greedy vs UCB1 vs TS on a 4-arm Bernoulli with $T = 500$ and 5 seeds. Tabulate final regret. Which algorithm has the lowest variance across seeds?

Ex. 6 ($\star\star$). Compute the discounted return of the reward sequence $(1, 1, 1)$ for $\gamma = 0.5$ by hand, then verify with code. Repeat for $\gamma = 1$ and the sequence $(0, 0, 10)$.

Ex. 7 ($\star\star$). Run LinUCB and ϵ-greedy on the synthetic insurance contextual bandit for $T = 1500$. Plot cumulative regret. The gap is the value of using context.

Ex. 8 ($\star\star\star$, open-ended). Implement the *Self-Normalised IPS* (SNIPS) off-policy estimator $\widehat{V}(\pi) = \sum_i w_i r_i / \sum_i w_i$. Use it to estimate the value of a candidate policy from logs of a different behaviour policy. Discuss when SNIPS is biased vs. when it is dangerous.

9 Further reading

The companion document `extended_reading.pdf` unpacks five papers in depth: Robbins (1952), Auer–Cesa-Bianchi–Fischer (2002), Agrawal–Goyal (2012), Li–Chu–Langford–Schapire (2010), and Russo–Van Roy (2014). The bandit dossier in `research/bandits_matching_sota.md` is the source material; this week's reading list is a curated entry point.